AWS Level 3 iteration 4 terminaldokumentation

Claudio Verdi | APL | 24 oktober 2026

Jag skapade en Ubuntu-instans med Terraform och använde user-data för att installera Ansible. En lokal playbook installerade Nginx och Git, hämtade webbplatsen och publicerade den automatiskt. Loggen och webbtesterna bekräftade resultatet. En andra Ansible-körning gav inga ändringar. Därefter tog jag bort instansen och kontrollerade statusen terminated.

Miljö och resurser

Region: eu-north-1

Instans: i-0037b3fe4cc3dfff8

Typ och operativsystem: t3.micro / Ubuntu 24.04 LTS

AMI: ami-06fb1802e2b757f09

Subnät: subnet-05d4dca4820d2eb94

Säkerhetsgrupp: sg-0fb87d0f08a213466

Nyckelpar: devops.school.level3.tabish-admin

Tidigare publik IP: 13.62.49.209

AWS provider: 6.70.0

Ansible och ansible-core: 9.2.0 / 2.16.3

Nginx: 1.24.0

Rotdisk: 8 GiB gp3 / encrypted / delete_on_termination=true

Referens: Level3 Iteration4(3).sh. Jag anpassade pakethantering och filsökvägar till Ubuntu och webbplatsens faktiska struktur.

Förberedelser och user-data

Jag skapade ~/aws-devops-level3-iteration4 och kopierade fem Terraform-filer från iteration 3 utan state-filer. Den befintliga säkerhetsgruppen och SSH-nyckeln återanvändes. Skriptet user-data.web.sh kontrollerades med bash -n.

User-data uppdaterar paketlistan, kör apt-get upgrade och installerar Ansible. Därefter skriver skriptet /opt/config-and-deploy.web.yml och kör playbooken lokalt på EC2-instansen.

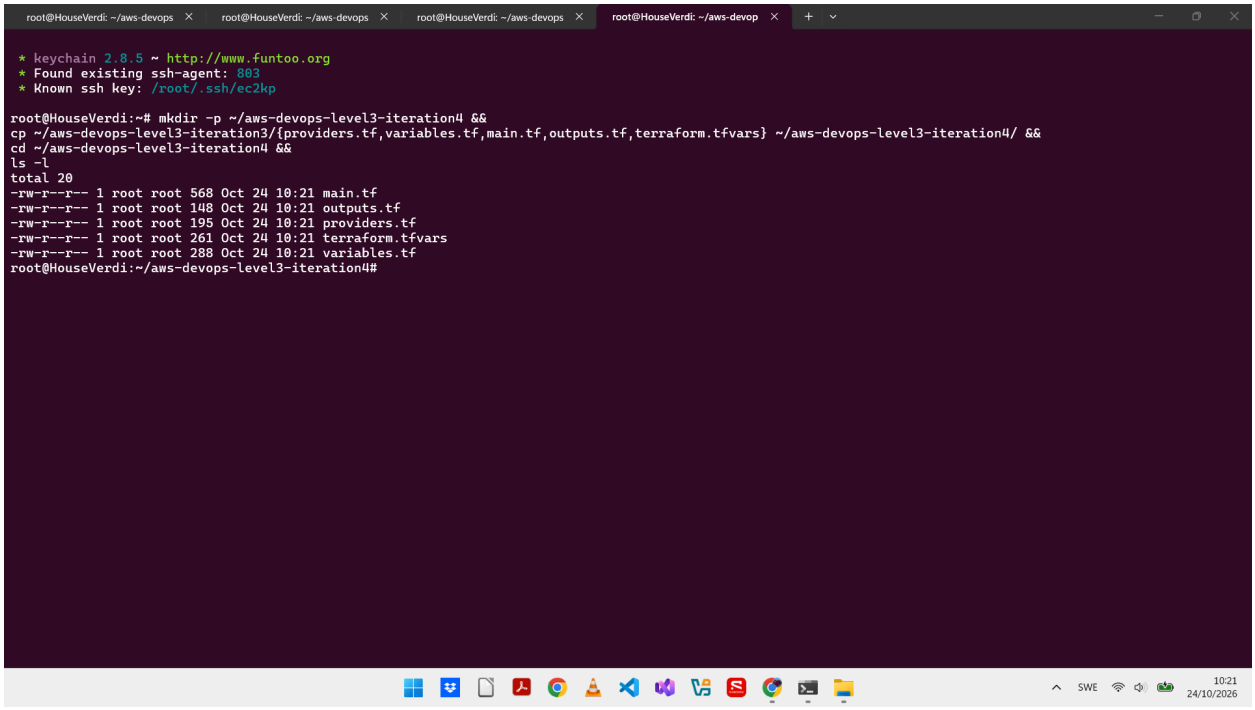


Bild 1. Fem Terraform-filer i den nya projektkatalogen

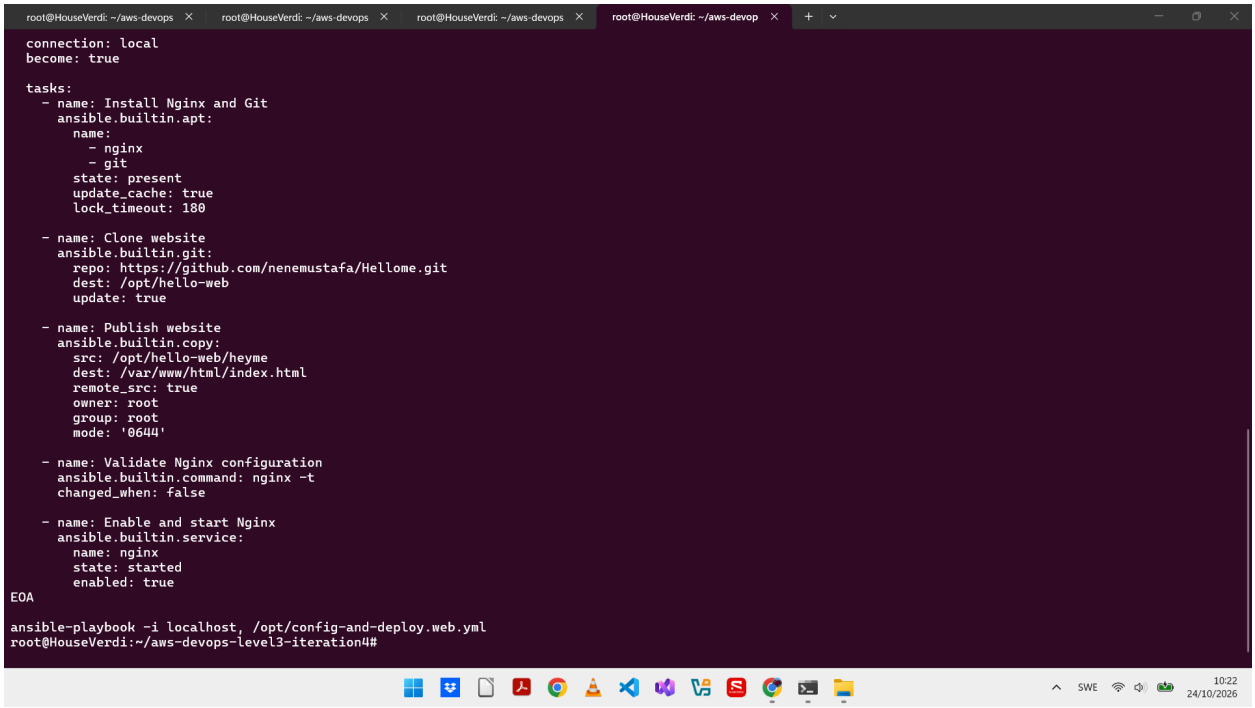


Bild 2. Synlig del av playbooken efter syntaxkontroll

Terraform validering och planering

terraform fmt, init och validate slutfördes utan fel. AWS-provider 6.70.0 installerades. main.tf läser user-data från fil, kräver IMDSv2 och krypterar rotdisken. Planen sparades som tfplan och visade en ny resurs.

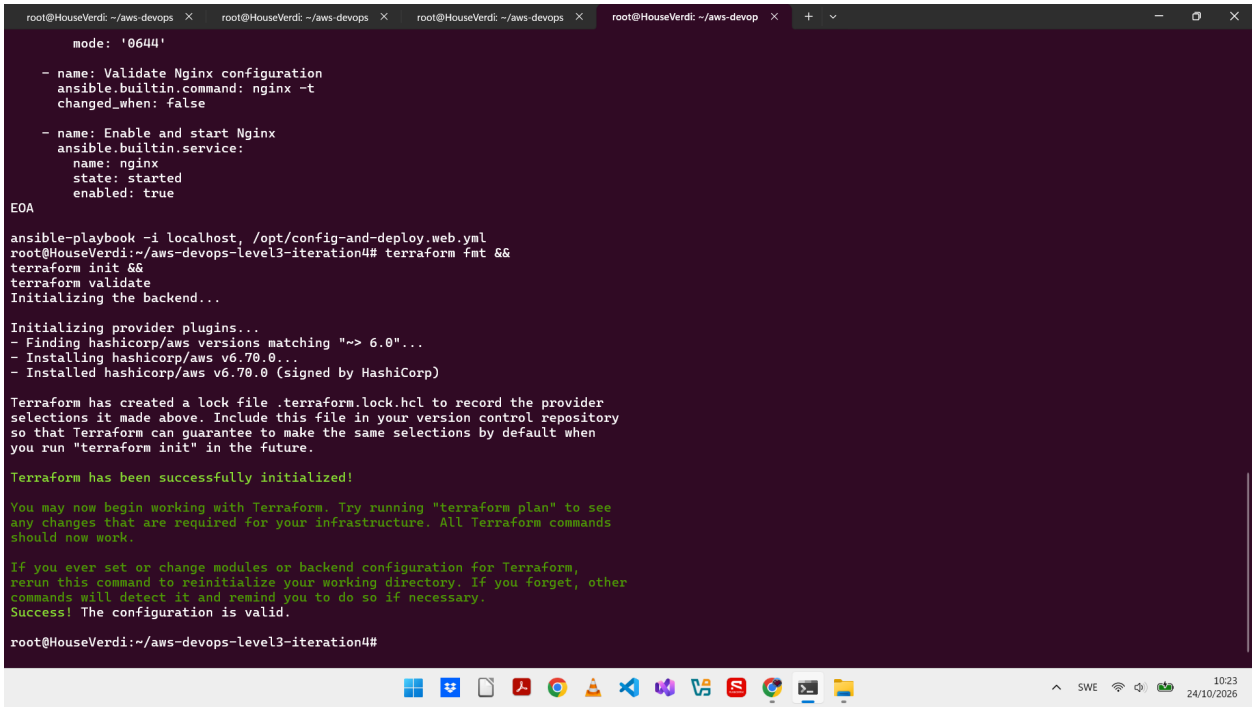


Bild 3. Initiering och lyckad validering

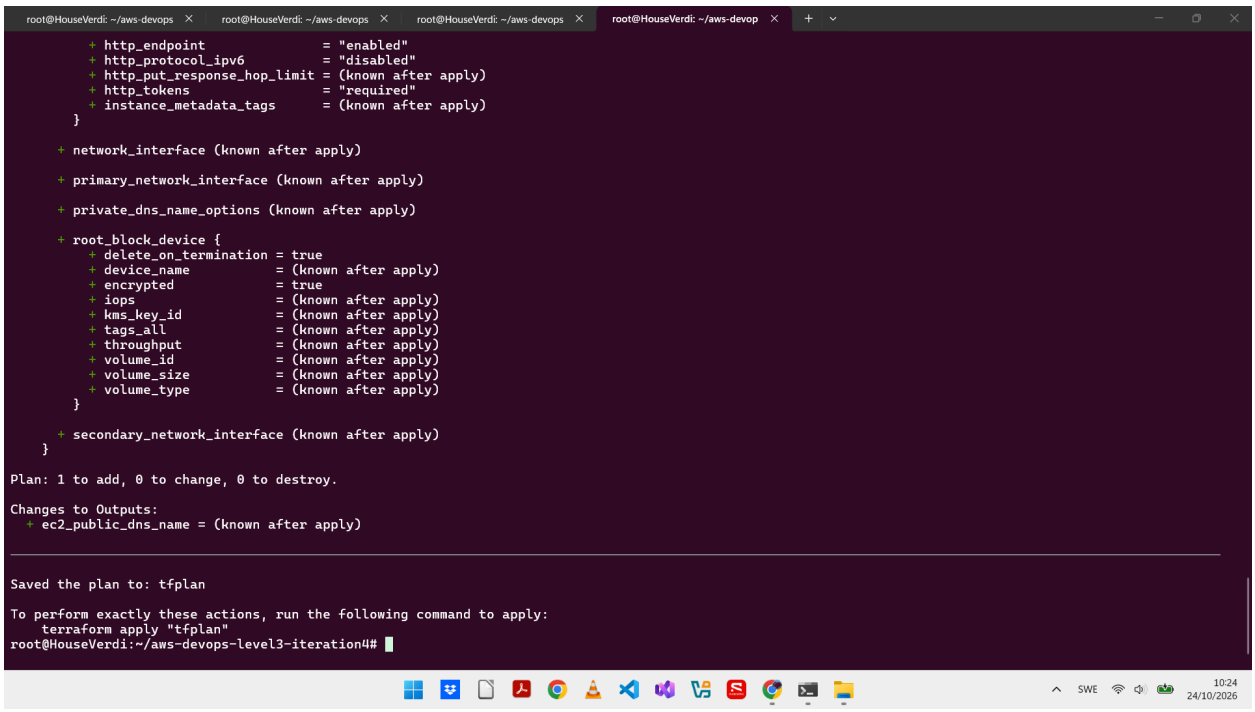


Bild 4. Plan med 1 att skapa och inga andra ändringar

Skapande och cloud-init

terraform apply tfplan skapade instansen på 13 sekunder. DNS-namnet blev ec2-13-62-49-209.eu-north-1.compute.amazonaws.com. Jag läste cloud-init-status via SSH som ubuntu med ~/.ssh/id_rsa_level3. Resultatet var status: done.

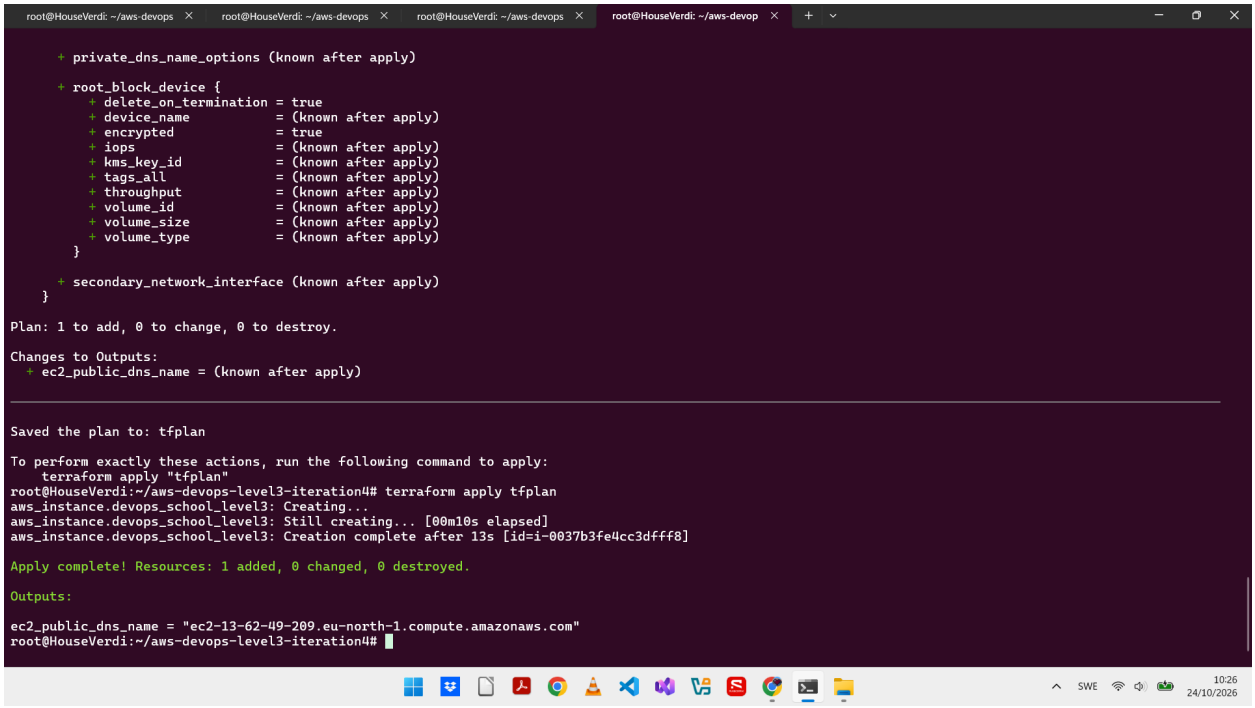


Bild 5. Instansen skapad med ID och DNS-namn

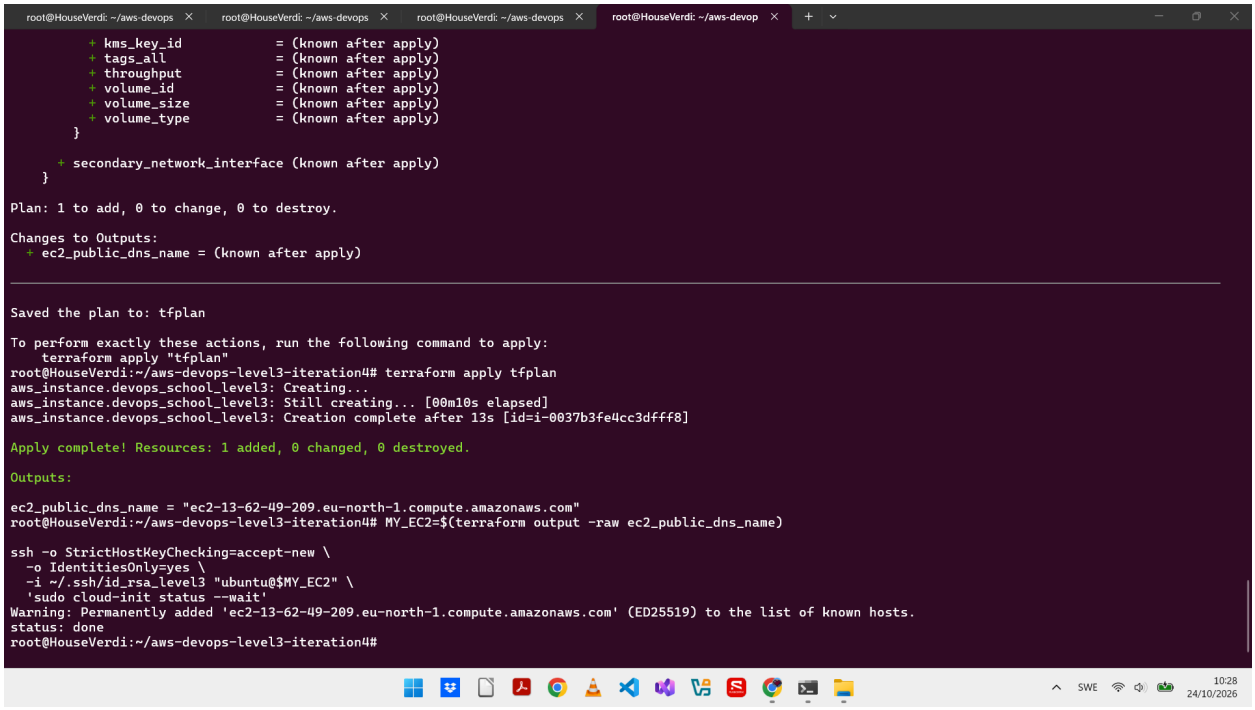


Bild 6. Cloud-init visar status done

Automatisk konfiguration med Ansible

Playbooken använder apt för Nginx och Git, git för hämtning till /opt/hello-web och copy med remote_src för att publicera heyme som /var/www/html/index.html. Den kontrollerar nginx -t och säkerställer att Nginx är startad och aktiverad.

PLAY RECAP visar ok=6, changed=3, unreachable=0 och failed=0. De tre ändringarna gällde paket, Git-hämtning och publicering. Cloud-init avslutades 08.27.47 UTC den 24 oktober 2026, motsvarande 10.27.47 svensk tid, efter 96,49 sekunders upptid.

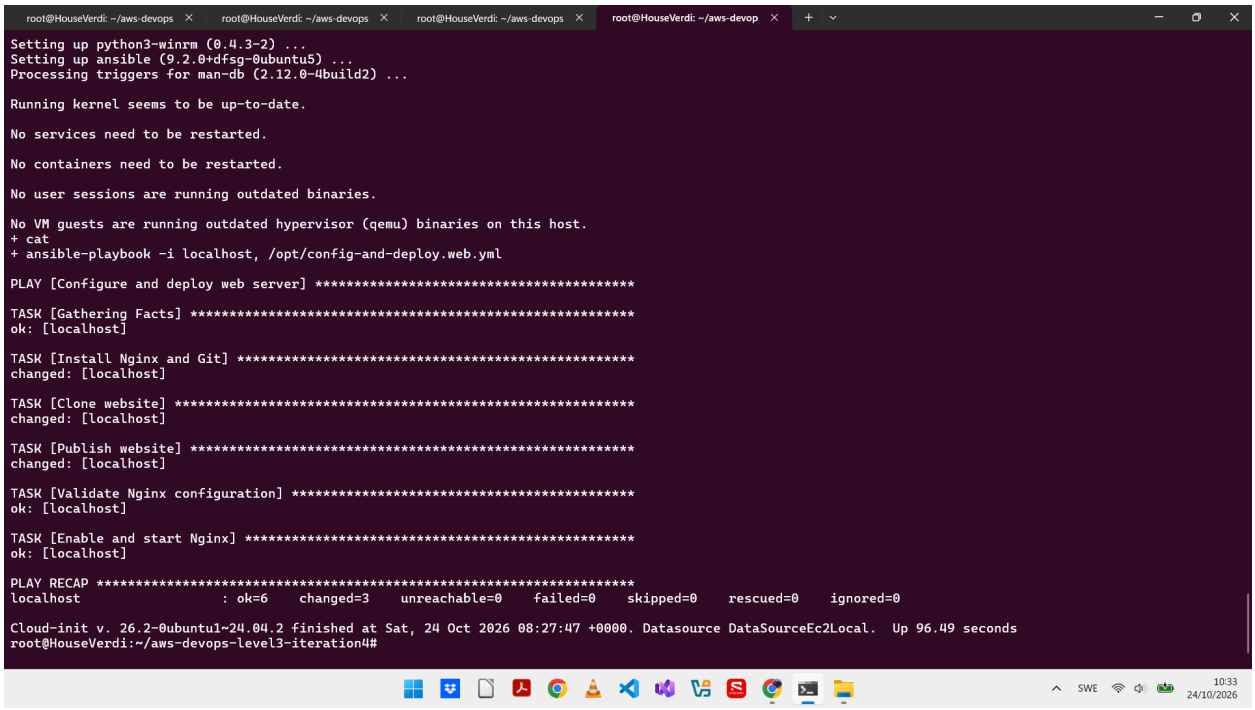


Bild 7. Första Ansible-körningen slutförd utan fel

Skillnaden mot iteration 3

I iteration 3 låg installation och publicering i Bash-kommandon. Här startar Bash en deklarativ Ansible-playbook som beskriver önskat tillstånd för paket, filer och tjänsten.

HTTP och kontroll av sidans innehåll

curl -I gav HTTP/1.1 200 OK, nginx/1.24.0 (Ubuntu), Content-Type text/html och Content-Length 499. Innehållstestet med den korrigerade söktexten Hey me! gav YES och slutkod 0.

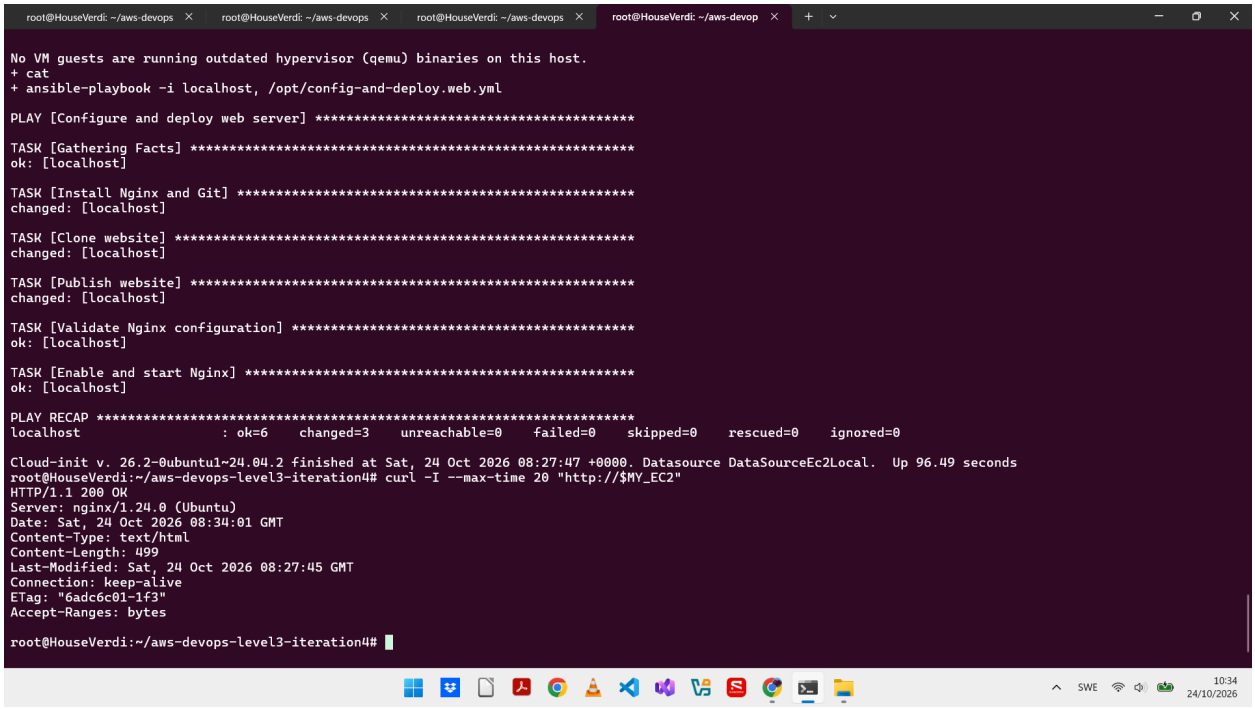


Bild 8. HTTP 200 från webbservern

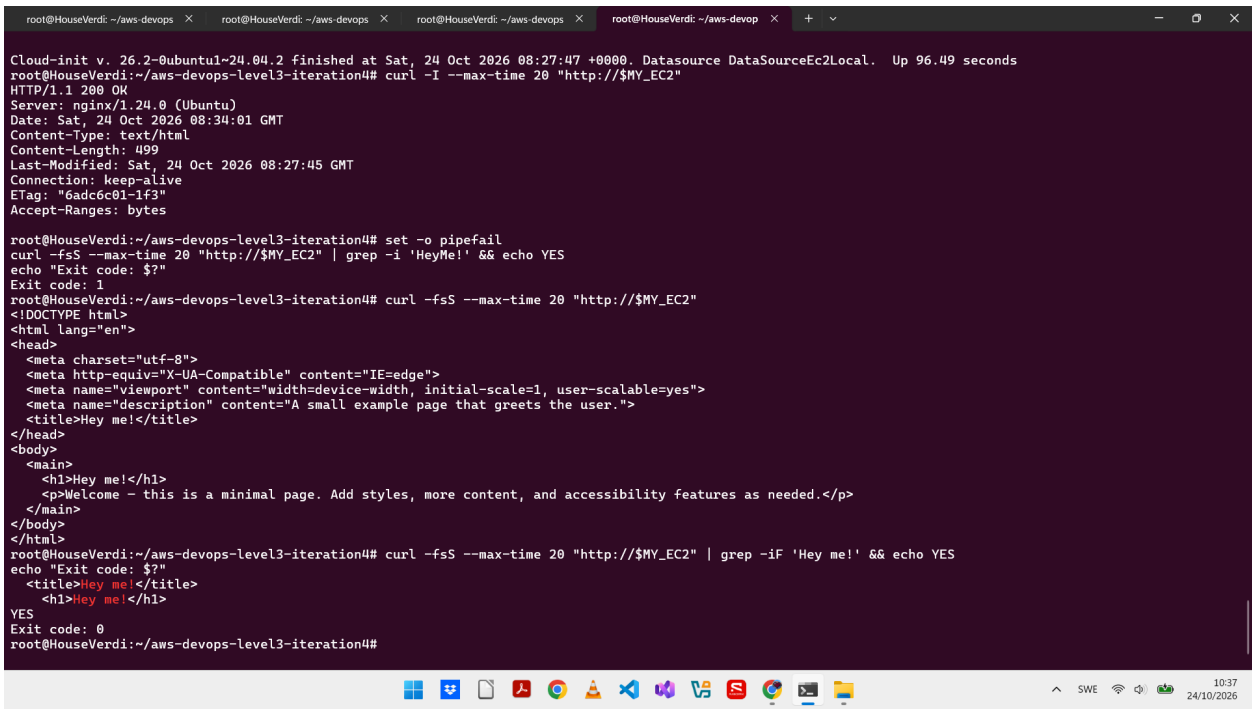


Bild 9. Korrigerat innehållstest med YES och slutkod 0

Webbläsare och kontroll av infrastrukturen

Webbläsaren visade Hey me! på instansens DNS-adress. terraform plan gav No changes. Terraform kontrollerar infrastrukturen; webbplatsens innehåll kontrollerades separat.

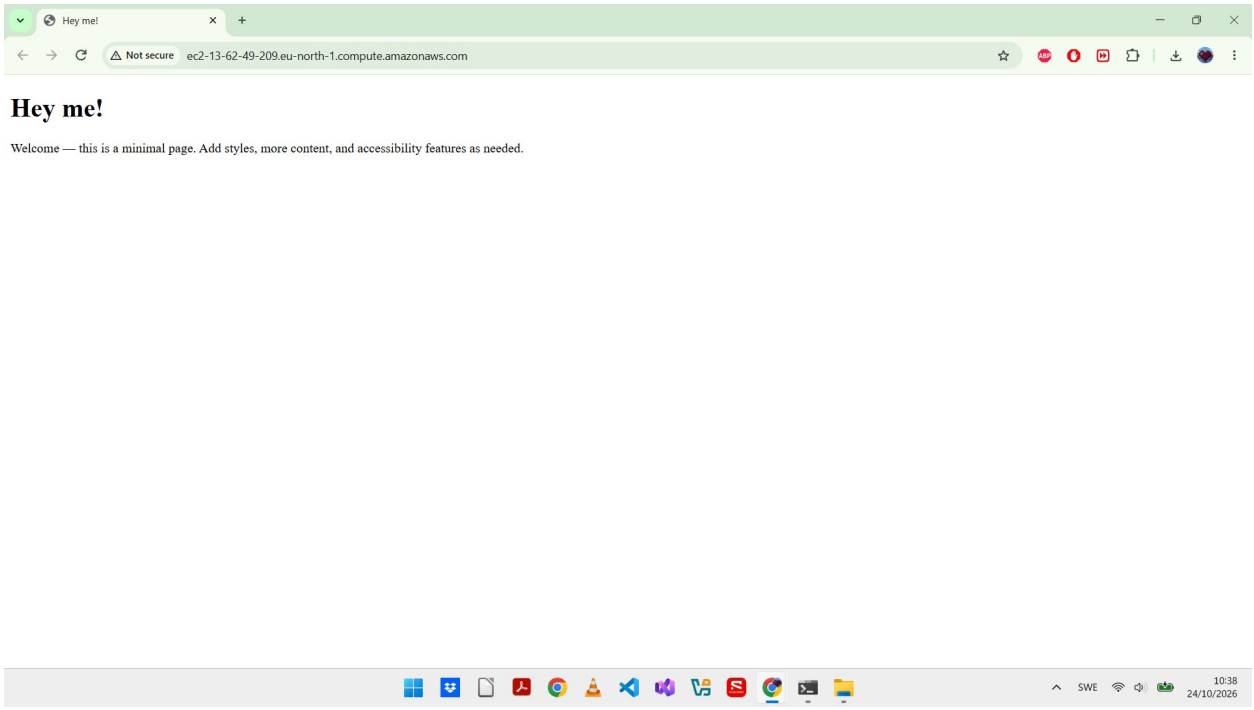


Bild 10. Webbplatsen på rätt serveradress

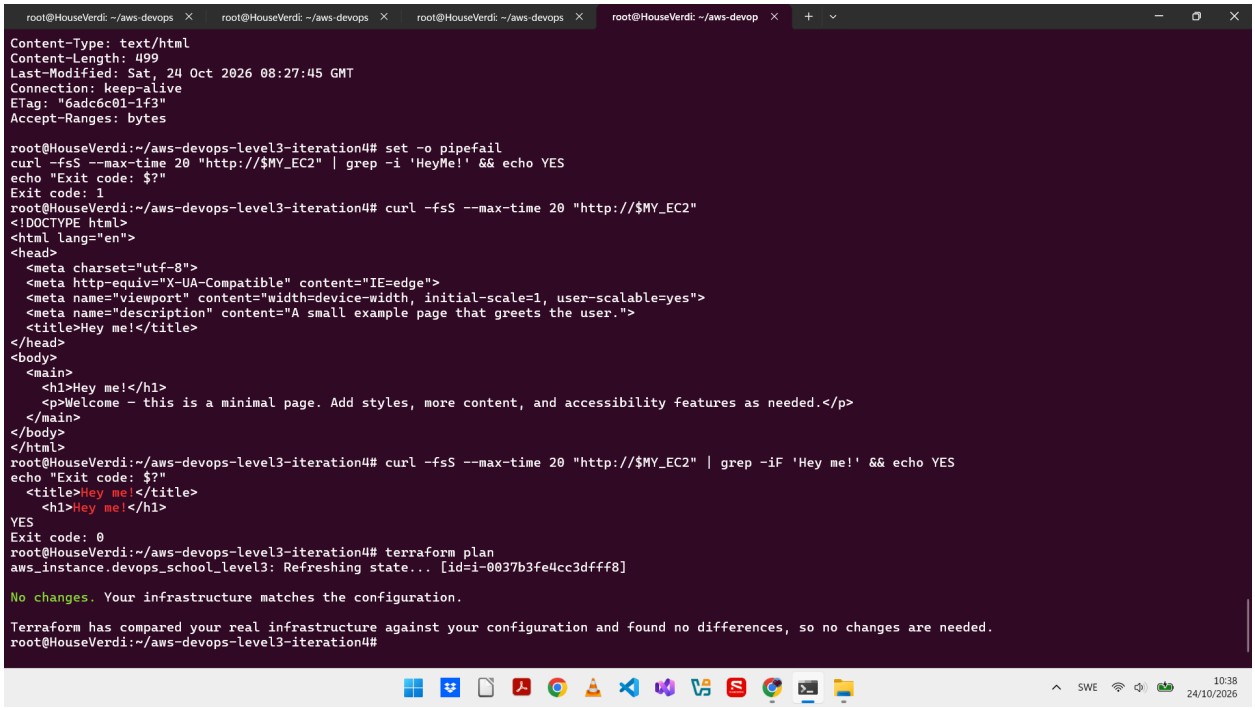


Bild 11. Terraform visar No changes

Andra Ansible körningen

Jag körde samma playbook igen via SSH. Alla sex moment gav ok och sammanfattningen blev ok=6, changed=0, unreachable=0, failed=0. Det visar att den andra körningen inte behövde ändra det befintliga tillståndet.

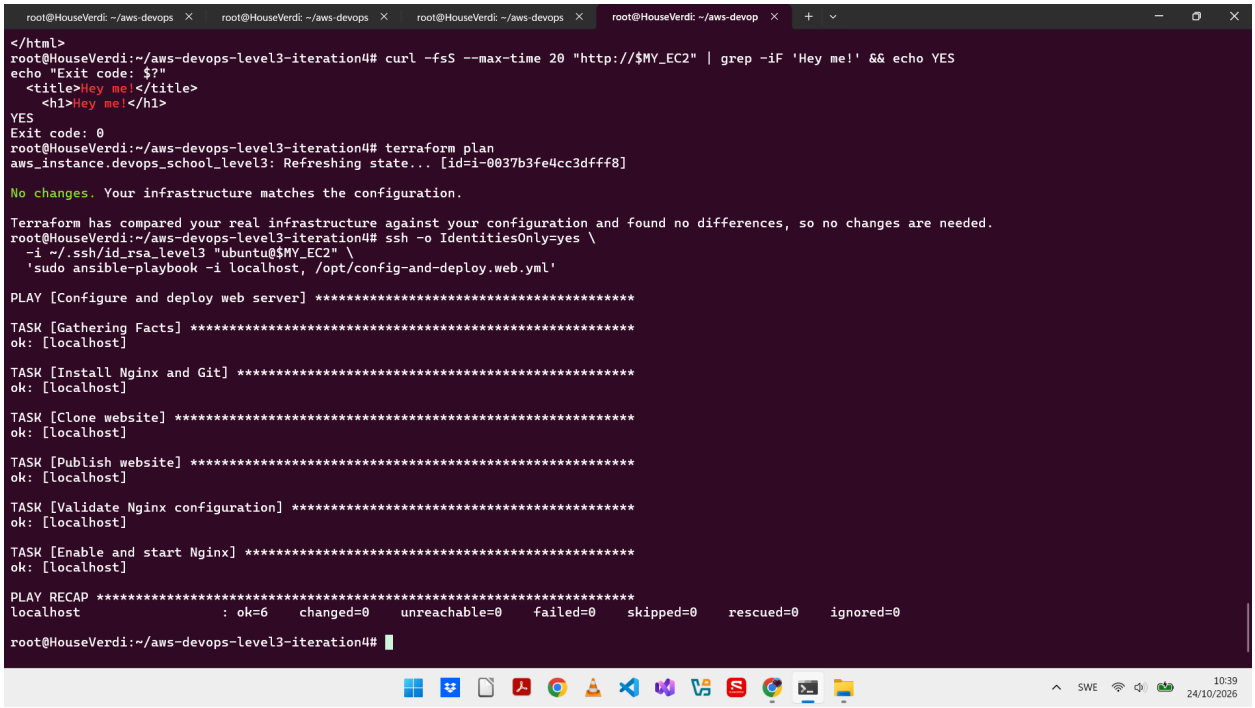


Bild 12. Andra körningen utan ändringar eller fel

Tolkning och begränsningar

Resultatet verifierar omkörning med dåvarande paket och Git-innehåll. update=true kan hämta senare ändringar från kodförrådet. nginx -t körs vid varje körning men rapporterar ingen ändring genom changed_when: false. Något omstartstest utfördes inte.

Avslut och verifierad borttagning

Jag sparade konfiguration, terminalhistorik och cloud-init-logg. terraform plan -destroy -out=destroy.tfplan visade en resurs att ta bort. terraform apply destroy.tfplan slutfördes efter 33 sekunder med 1 destroyed. AWS CLI bekräftade terminated den 24 oktober kl. 10.44 svensk tid.

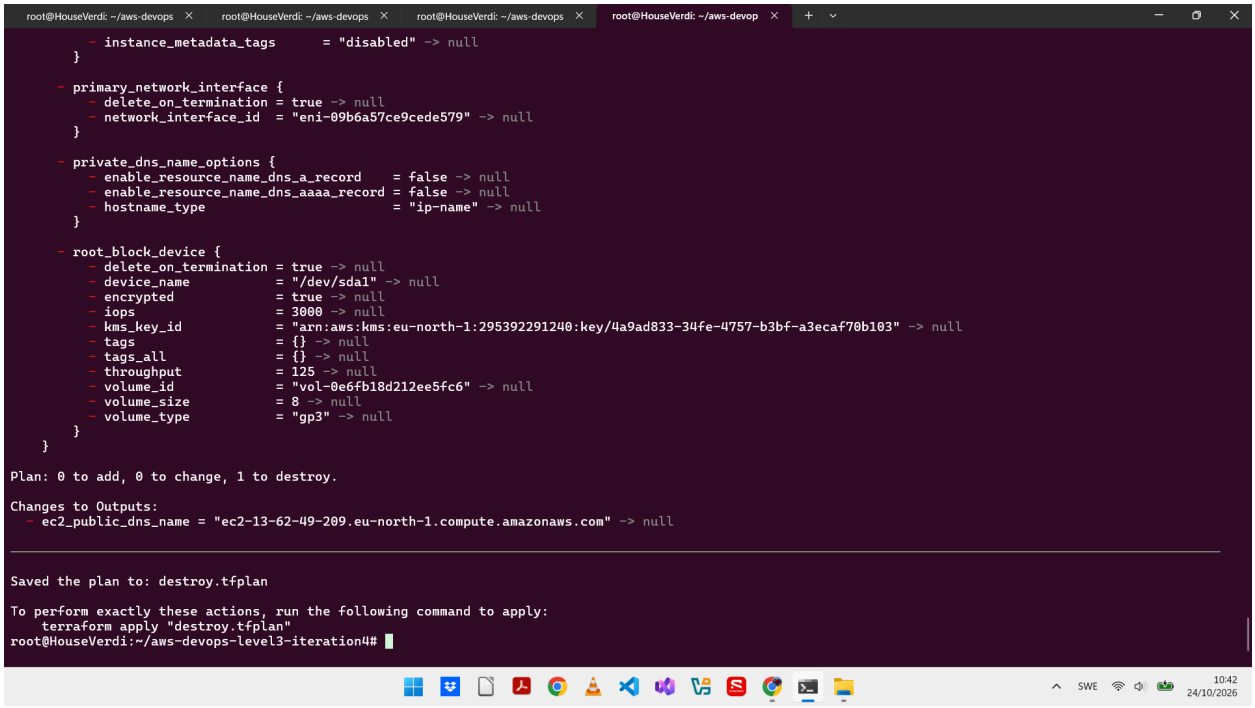


Bild 13. Borttagningsplan och rotdiskens inställningar

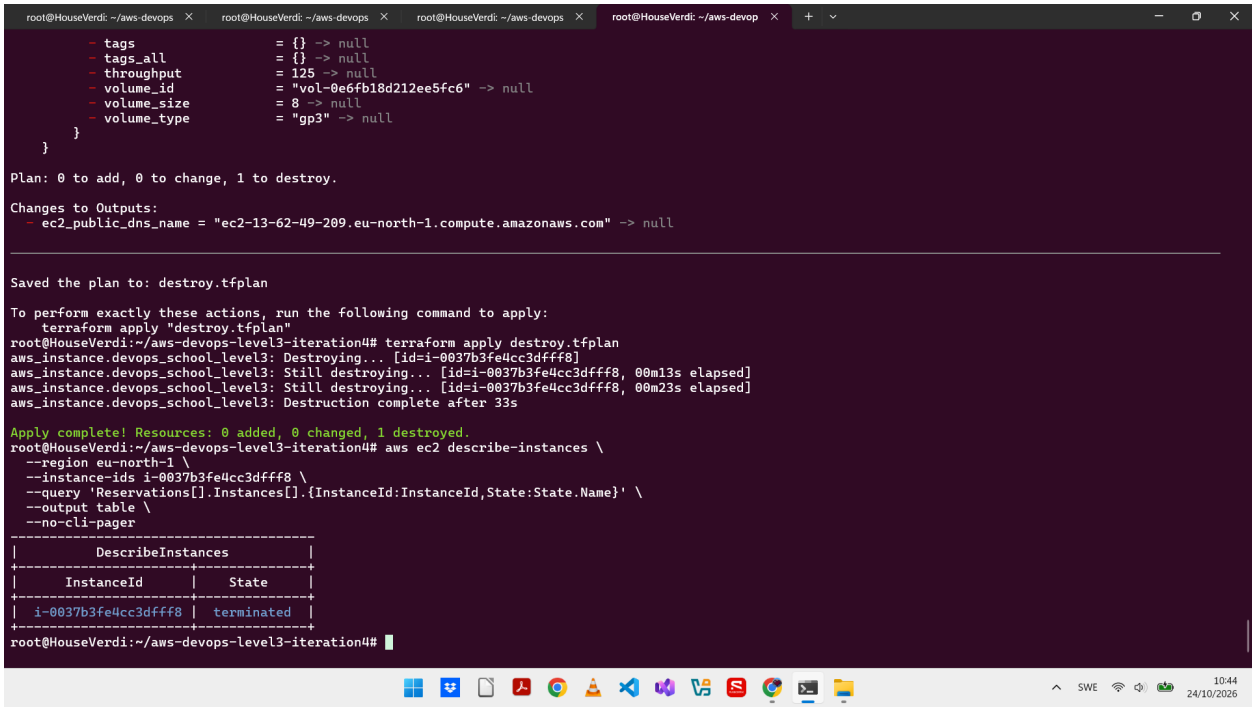


Bild 14. Slutförd borttagning och status terminated

Felsökning anpassningar och slutsats

Innehållstestets söktext

Den första sökningen efter HeyMe! gav slutkod 1. Jag läste HTML med curl och såg Hey me! med mellanslag. När söktexten korrigerades gav testet YES och slutkod 0. Webbplatsen behövde inte ändras; felet fanns i testets söktext.

Anpassningar från referensen

Guidens yum ersattes med apt och apt-get för Ubuntu. SSH-användaren blev ubuntu. I stället för synchronize från en html-katalog används copy med remote_src för filen heyme. Målet är /var/www/html/index.html. Explicit inventory localhost, och connection: local gör att playbooken körs lokalt på EC2-instansen.

Resultat och förbättringar

Terraform, user-data och Ansible skapade en fungerande webbserver med automatisk publicering. HTTP, innehållstest, webbläsare och logg bekräftar resultatet. En andra Ansible-körning gav inga ändringar. Möjliga förbättringar är HTTPS och låsning till en bestämd Git-commit för reproducerbart innehåll.

Rotdisken hade delete_on_termination=true. Befintligt nyckelpar och säkerhetsgrupp hanterades utanför Terraform och omfattades inte av borttagningen. Rapportens IP och DNS avser tiden före avslutet.

Underlag

Level3 Iteration4(3).sh; level3-4-konfiguration.txt; uppdaterad level3-4-terminal-history(1).txt; level3-4-cloud-init.log och infogade skärmbilder. Terminalhistoriken visar kommandon; resultaten styrks av skärmbilderna och serverloggen.

Bilaga Terraform grundfiler

providers.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

variables.tf

```
variable "aws_region" {
  type = string
}

variable "ami_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "main_sg_id" {
  type = string
}

variable "ec2_type" {
  type = string
}

variable "ec2_name" {
  type = string
}

variable "keypair" {
  type = string
}
```

Bilaga instans och variabelvärden

main.tf

```
resource "aws_instance" "devops_school_level3" {
  ami                         = var.ami_id
  instance_type               = var.ec2_type
  subnet_id                   = var.subnet_id
  key_name                    = var.keypair
  vpc_security_group_ids      = [var.main_sg_id]
  associate_public_ip_address = true

  user_data                   = file("${path.module}/user-data.web.sh")
  user_data_replace_on_change = true

  metadata_options {
    http_tokens = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = var.ec2_name
  }
}
```

outputs.tf

```
output "ec2_public_dns_name" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.devops_school_level3.public_dns
}
```

terraform.tfvars

```
aws_region = "eu-north-1"
ami_id     = "ami-06fb1802e2b757f09"
subnet_id  = "subnet-05d4dca4820d2eb94"
main_sg_id = "sg-0fb87d0f08a213466"
ec2_type   = "t3.micro"
ec2_name   = "devops-school-level3.tabish-admin"
keypair    = "devops.school.level3.tabish-admin"
```

Bilaga user-data och Ansible

```
#!/bin/bash
set -euxo pipefail
export DEBIAN_FRONTEND=noninteractive

apt-get update
apt-get upgrade -y
apt-get install -y ansible

cat > /opt/config-and-deploy.web.yml <<'EOA'
---
- name: Configure and deploy web server
  hosts: localhost
  connection: local
  become: true

  tasks:
    - name: Install Nginx and Git
      ansible.builtin.apt:
        name:
          - nginx
          - git
        state: present
        update_cache: true
        lock_timeout: 180

    - name: Clone website
      ansible.builtin.git:
        repo: https://github.com/nenemustafa/Hellome.git
        dest: /opt/hello-web
        update: true

    - name: Publish website
      ansible.builtin.copy:
        src: /opt/hello-web/heyme
        dest: /var/www/html/index.html
        remote_src: true
        owner: root
        group: root
        mode: '0644'

    - name: Validate Nginx configuration
      ansible.builtin.command: nginx -t
      changed_when: false

    - name: Enable and start Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
EOA

ansible-playbook -i localhost, /opt/config-and-deploy.web.yml
```

Bilaga kontroll och avslut

```
MY_EC2=$(terraform output -raw ec2_public_dns_name)
ssh -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 \
  "ubuntu@$MY_EC2" 'sudo cloud-init status --wait'
ssh -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 \
  "ubuntu@$MY_EC2" \
  'sudo tail -n 60 /var/log/cloud-init-output.log'
curl -I --max-time 20 "http://$MY_EC2"
set -o pipefail
curl -fsS --max-time 20 "http://$MY_EC2" |
  grep -iF 'Hey me!' && echo YES
echo "Exit code: $?"
terraform plan
ssh -o IdentitiesOnly=yes -i ~/.ssh/id_rsa_level3 \
  "ubuntu@$MY_EC2" \
  'sudo ansible-playbook -i localhost, /opt/config-and-deploy.web.yml'
```

Borttagning

```
terraform plan -destroy -out=destroy.tfplan
terraform apply destroy.tfplan
aws ec2 describe-instances --region eu-north-1 \
  --instance-ids i-0037b3fe4cc3dfff8 \
  --query 'Reservations[].Instances[].{InstanceId:InstanceId,State:State.Name}' \
  --output table --no-cli-pager
```